

The Open Source Audit

A Capstone Project for OSS NGMC Course
VIT Bhopal University | Open Source Software

Software Audited	Git
License	GNU General Public License v2 (GPL v2)
Category	Version Control System
Course	Open Source Software Units 1 – 5
Student Name	Kunjesh

Every tool you will use in your career was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice is not a footnote to your technical education — it is the foundation of it.

1. Introduction

This report presents a structured audit of Git, one of the most influential pieces of open-source software ever written. Git is a distributed version control system created by Linus Torvalds in April 2005 and released under the GNU General Public License version 2 (GPL v2). It is now the standard tool for version control in virtually every software project in the world — from personal experiments to the Linux kernel itself.

This audit examines Git not just as a technical tool, but as a philosophical statement about how software should be built and shared. The report is structured across five parts: the origin and philosophy of Git (Part A), its Linux footprint (Part B), the FOSS ecosystem it belongs to (Part C), a comparison with proprietary alternatives (Part D), and five shell scripts that demonstrate practical Linux skills connected to the open-source workflow.

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

To understand why Git exists, you have to understand what version control looked like in 2005 — and why Linus Torvalds, the creator of Linux, decided none of it was good enough.

The Linux kernel in 2002 began using a proprietary distributed version control system called BitKeeper. BitKeeper was free-of-charge for open-source projects, but it was not free as in freedom — it was owned by a company called BitMover, and its source code was closed. The arrangement was controversial from the start: many in the free software community objected to the Linux kernel, the very symbol of open development, depending on a proprietary tool.

In April 2005, the fragile arrangement collapsed. BitMover revoked the free license after discovering that a Linux kernel developer had written a tool to interoperate with BitKeeper — a violation of the license terms. The Linux project lost its version control system almost overnight. The community evaluated alternatives: CVS was old and slow, Subversion was centralised and not well-suited to the distributed nature of kernel development, and no other free tool met the performance requirements of a project with thousands of contributors.

Linus Torvalds, unwilling to use any existing tool, spent ten days writing Git from scratch. His design goals were uncompromising: speed above all, distributed architecture with no single point of failure, strong protection against data corruption, and full support for non-linear development with thousands of parallel branches. Git was designed to handle the Linux kernel — one of the largest and most complex software projects in existence — which meant it was overengineered for almost everything else. That over-engineering is exactly why it became universal.

The broader lesson from Git's origin is important: it was born because a proprietary tool failed the community that depended on it. The lesson Torvalds drew — and encoded into Git's license — was that the tools of collaborative development must themselves be open. You cannot build a free operating system on a foundation you do not control.

A2. The License — What It Actually Says

Git is licensed under the GNU General Public License version 2 (GPL v2). Understanding what this license actually does requires understanding the four freedoms that define free software, as articulated by Richard Stallman and the GNU Project:

- **Freedom 0:** The freedom to run the program for any purpose.
- **Freedom 1:** The freedom to study how the program works and change it — which requires access to the source code.
- **Freedom 2:** The freedom to redistribute copies.
- **Freedom 3:** The freedom to distribute modified versions, so the whole community can benefit.

GPL v2 grants all four freedoms. It also enforces them through a concept called copyleft: if you distribute a modified version of Git, you must release your modifications under the same GPL v2 license. You cannot take Git, make improvements, and keep those improvements proprietary. This is the key structural difference between GPL and more permissive licenses like MIT.

Can a company take Git, modify it, and sell it without sharing changes? No. GPL v2 is explicit: distribution of a modified version requires releasing the source code under the same terms. A company can sell support, hosting, or services built around Git — but not a closed-source fork. This is exactly why Linus chose GPL v2: he wanted to guarantee that no company could privatise the kernel's tools the way BitMover had privatised BitKeeper.

GPL v2 vs MIT: MIT is a permissive license — it allows anyone to take the code, modify it, and release it under any terms, including proprietary ones. A company could fork an MIT-licensed project, add features, and sell a closed-source version without ever contributing back. GPL prevents this. If I were choosing a license for something I built and cared about, I would choose GPL v2 or GPL v3 if I wanted to guarantee that my work stayed free. I would choose MIT if I wanted maximum adoption and did not mind companies building proprietary products on top of my code.

Free as in beer vs free as in freedom: BitKeeper was free as in beer — it cost nothing to download and use, but the source code was locked and the license could be revoked. Git is free as in freedom — not only does it cost nothing, but you have the right to study it, change it, and share it. The difference is the difference between a gift and a right.

A3. The Ethics of Open Source

Should all software be open source? There is a strong argument for it: transparency enables security (anyone can audit the code for vulnerabilities), collective effort accelerates progress, and treating software as a public good prevents monopolisation of essential infrastructure. When the software running elections, hospitals, or financial systems is closed, citizens have no ability to verify or contest what it does.

The argument against is also serious: some software requires significant investment to develop, and without the ability to monetise through licensing, the incentive to build it shrinks. Niche professional tools — specialised medical imaging software, for instance — might simply not exist if there were no closed-source business model to fund their development. The question is not binary; it is about which parts of the software stack should be treated as infrastructure and which as products.

Is it ethical for companies to profit from free community labor? Companies like Red Hat, Google, and Meta collectively employ thousands of engineers who contribute to Linux and other open-source projects — and they also extract enormous value from those projects. The ethical calculus is complicated. Red Hat's model — contributing back heavily while selling support and enterprise distributions — is widely seen as legitimate. Meta using Linux to run billions of dollars worth of infrastructure while contributing back comparatively little raises harder questions. The GPL tries to address this structurally: you must share modifications if you

distribute the software. But using software internally without distributing it technically requires no contribution — a tension that gave rise to the GNU Affero General Public License (AGPL), designed specifically for networked services.

Standing on the shoulders of giants: In software, almost nothing is built from nothing. Every compiler, every operating system, every web framework rests on decades of prior work — much of it open source. Git was possible because C existed, because Unix existed, because the GNU tools existed. Open source does not undermine original innovation; it accelerates it by eliminating the need to rebuild foundations. The question is whether the people building on those foundations acknowledge their debt and contribute back.

Part B — Linux Footprint

Installation

Git is available through virtually every Linux package manager. On Debian/Ubuntu systems, installation requires a single command:

```
$ sudo apt install git
```

On Fedora/RHEL/CentOS systems, the equivalent is:

```
$ sudo dnf install git
```

Git can also be compiled from source, which is common when the latest version is needed or when installing on a system without a package manager. The source is available at <https://git-scm.com/download/linux> and on the official GitHub mirror.

Key Directories

Path	Contents	Purpose
/usr/bin/git	Main executable	The primary Git binary called for all operations
/usr/lib/git-core/	Sub-commands	Individual executables for each Git sub-command (git-push, git-pull, etc.)
/usr/share/git-core/	Shared data	Templates and helper scripts for new repositories
/etc/gitconfig	System config	System-wide Git configuration applied to all users
~/.gitconfig	User config	Per-user configuration: name, email, aliases, preferences
<repo>/.git/	Repo data	All version history, objects, refs, and hooks for a repository

User and Group

Unlike a server daemon such as Apache or MySQL, Git does not run as a dedicated system user in normal usage — it runs as the user who invokes it. This is appropriate because Git is a user-space tool, not a background service. The git binary itself is owned by root and world-executable:

```
-rwxr-xr-x 1 root root /usr/bin/git
```

When Git is used in a server context (e.g., hosting repositories via SSH), a dedicated 'git' system user is typically created. This limits the attack surface: even if the git user's shell is compromised, it cannot escalate privileges to root. This principle of least privilege is a core Linux security concept.

Service Management

Git itself is not a service — it is invoked as a command. However, the Git daemon (which allows read-only access to repositories over the git:// protocol) can be run as a systemd service:

```
$ sudo systemctl start git.service
$ sudo systemctl stop git.service
$ sudo systemctl enable git.service # Start on boot
```

```
$ sudo systemctl status git.service
```

Update Model

Security patches for Git are pushed through the standard package manager update pipeline. The Git project maintains a security mailing list and issues CVEs for vulnerabilities. When a patch is released, it is packaged by distribution maintainers and delivered via the normal update mechanism:

```
$ sudo apt update && sudo apt upgrade git
```

This is the open-source security model in action: because the source code is public, security researchers worldwide can find vulnerabilities and contribute patches. The fix is then visible and auditable before it reaches your system.

Part C — The FOSS Ecosystem

Dependencies

Git's dependency footprint is intentionally minimal — this was a deliberate design decision by Torvalds, who wanted a tool that could be compiled and run on almost any Unix-like system. Its core dependencies include:

- **C standard library (glibc):** Git is written in C, and links against glibc for standard I/O and system calls.
- **zlib:** Used for compression of object data in the Git object store.
- **OpenSSL / libcrypto:** Used for SHA-1 (and increasingly SHA-256) hashing of objects.
- **libcurl:** For HTTP/HTTPS transport when pushing and pulling from remote repositories.
- **Perl:** Several Git utilities and helper scripts are written in Perl.
- **Python:** Some Git add-ons and hooks are Python-based.

What Git Has Enabled

Git's impact on the software ecosystem is arguably larger than any other single tool created in the last two decades. The following are direct consequences of Git's existence:

- **GitHub (2008):** Built on top of Git, GitHub became the largest code hosting platform in the world, with over 100 million repositories. It changed how open-source collaboration works.
- **GitLab and Bitbucket:** Competing platforms also built on Git, offering self-hosted alternatives.
- **CI/CD as a discipline:** Git's branch model made continuous integration pipelines practical. Tools like Jenkins, GitHub Actions, and GitLab CI are built around Git events (push, pull request, merge).
- **Infrastructure as Code:** Tools like Terraform, Ansible, and Kubernetes manifests are stored and versioned in Git. The concept of GitOps — where the state of infrastructure is defined by what is in a Git repository — would not exist without Git.
- **Open-source contribution model:** The pull request / merge request workflow, pioneered by GitHub on top of Git, became the standard model for open-source contribution globally.

Community and Governance

Git is governed by a loose meritocracy centered on Junio C Hamano, who has been the maintainer since Linus Torvalds handed off the project in July 2005 — just a few months after creating it. The project is developed on a public mailing list (git@vger.kernel.org) rather than through pull requests on a web interface. Patches are submitted via email, reviewed publicly, and applied by the maintainer. This old-school workflow reflects the project's Unix heritage and prioritises signal over noise.

Part D — Open Source vs Proprietary

The closest proprietary alternative to Git is Perforce Helix Core — a centralised version control system used primarily in game development and large enterprise environments. A second comparison point is Microsoft Team Foundation Version Control (TFVC), now mostly supplanted by Azure DevOps with Git underneath.

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free. No licensing fees ever.	Expensive. Enterprise licensing can run thousands of dollars per year.
Security	Source code is fully auditable by anyone. Vulnerabilities are covered and patched publicly.	Security relies on vendor-assessed controls. Customers cannot audit the code.
Support	Community support via mailing list and Stack Overflow. Commercial support is available.	Dedicated vendor support is available. GitHub Enterprise support is also available.
Freedom to Modify	Full. GPL v2 allows any modification for any purpose.	None. Source code is closed; modifications are not permitted.
Community Control	Governed by community consensus and maintained by a small group of volunteers.	Controlled entirely by Perforce Software Inc. The roadmap is driven by the company.
Performance at Scale	Excellent for most projects. Can struggle with very large binary files (large history assets, etc.).	Strong for large binary files (large history assets, etc.).
Ecosystem	Enormous. GitHub, GitLab, every CI/CD tool, every IDE, and a vast ecosystem of integrations.	Smaller. Dependent on game and enterprise tools but no cross-industry ecosystem.

Verdict

For the overwhelming majority of real-world deployments — from individual developers to large engineering organisations — Git is the unambiguous recommendation. Its cost (zero), ecosystem (unmatched), and flexibility (branches, distributed architecture, hooks, integrations) make it the correct choice for virtually every software project. The only genuine case for Perforce over Git is in environments with very large binary assets — game studios storing multi-gigabyte texture files and build artifacts — where Perforce's centralised model and strong binary locking have real advantages.

Would I contribute to Git? Yes, unreservedly. Git is one of the most used software tools in human history, and it remains maintained by a small community of volunteers and employed engineers. Contributing — even documentation improvements or small bug fixes — is a direct contribution to the infrastructure of global software development. The mailing-list workflow has a steep learning curve, but the impact-per-contribution ratio is exceptional.

Part E — Shell Scripts

Script 1 — System Identity Report

Shell Concepts Used: Variables, echo, command substitution (\$()), output formatting

Explanation:

This script introduces the Linux system like a welcome screen. It uses command substitution to capture the output of `uname -r`, `whoami`, `uptime -p`, `date`, and reading `/etc/os-release` into variables. The information is then displayed in a formatted table-style output using `echo`. It demonstrates how shell variables and command substitution combine to produce dynamic output — the same principle underlying all shell automation.

Script Code:

```
#!/bin/bash

# Script 1: System Identity Report
# Author: Kunjesh | Course: Open Source Software

STUDENT_NAME="Kunjesh"
SOFTWARE_CHOICE="Git"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%d %B %Y %H:%M:%S')
DISTR0=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d "'")

echo "====="
echo " Open Source Audit — $STUDENT_NAME"
echo " Software: $SOFTWARE_CHOICE"
echo "====="
echo " Distribution : $DISTR0"
echo " Kernel : $KERNEL"
echo " User : $USER_NAME"
echo " Home Dir : $HOME_DIR"
echo " Uptime : $UPTIME"
echo " Date & Time : $CURRENT_DATE"
echo " OS License : GPL v2"
echo "=====
```

Script 2 — FOSS Package Inspector

Shell Concepts Used: if-then-else, case statement, dpkg/rpm with grep, pipe

Explanation:

This script checks whether Git is installed using the package manager available on the system (dpkg for Debian/Ubuntu, rpm for Fedora/RHEL). It uses a pipe with grep to filter the output to only show relevant fields. A case statement maps package names to philosophy notes — demonstrating how case statements provide clean multi-branch logic. The if-then-else handles the installed/not-installed decision cleanly.

Script Code:

```
#!/bin/bash

# Script 2: FOSS Package Inspector
PACKAGE="git"

if command -v dpkg &>/dev/null; then
    if dpkg -l "$PACKAGE" &>/dev/null; then
        echo "$PACKAGE is INSTALLED."
        VERSION=$(git --version | awk '{print $3}')
        echo "Version: $VERSION | License: GPL v2"
    else
        echo "$PACKAGE is NOT installed. sudo apt install git"
    fi
fi

case $PACKAGE in
    git) echo "Git: Born when proprietary VCS failed the Linux community." ;;
    httpd) echo "Apache: The server that built the open internet." ;;
    mysql) echo "MySQL: Open source at the heart of millions of apps." ;;
    python3) echo "Python: Shaped entirely by community consensus." ;;
    *) echo "$PACKAGE: Contributing to the free software ecosystem." ;;
esac
```

Script 3 — Disk and Permission Auditor

Shell Concepts Used: for loop, ls -ld, du -sh, awk, cut

Explanation:

This script loops through a predefined array of important system directories using a for loop, checking each one exists before auditing it. For each directory, it uses ls -ld piped through awk to extract the permissions, owner, and group fields, and du -sh with cut to get the human-readable size. It then checks specifically for Git's configuration files, demonstrating how a general-purpose loop can be combined with software-specific checks.

Script Code:

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

for DIR in "${DIRS[@]}; do
if [ -d "$DIR" ]; then
PERMS=$(ls -ld "$DIR" | awk '{print $1}')
OWNER=$(ls -ld "$DIR" | awk '{print $3}')
SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
echo "$DIR => $PERMS | Owner: $OWNER | Size: $SIZE"
else
echo "$DIR => does NOT exist"
fi
done

# Check Git config directory
if [ -f "$HOME/.gitconfig" ]; then
PERMS=$(ls -l "$HOME/.gitconfig" | awk '{print $1, $3, $4}')
echo "Git user config found: $PERMS"
fi
```

Script 4 — Log File Analyzer

Shell Concepts Used: while read loop, if-then, counter variables, command-line arguments (\$1, \$2), tail + grep

Explanation:

This script accepts a log file path and optional keyword as command-line arguments (\$1 and \$2), with a default keyword of 'error'. It validates the input using if-then, then reads the file line by line with a while-read loop. Each line is checked for the keyword using grep -iq (case-insensitive). A counter variable tracks the number of matches. After the loop, it prints a summary and uses tail + grep to show the last 5 matching lines. A do-while style retry loop tries alternative log file locations if the specified file is not found.

Script Code:

```
#!/bin/bash

# Script 4: Log File Analyzer

# Usage: ./script4_log_analyzer.sh /var/log/syslog [KEYWORD]

LOGFILE=$1

KEYWORD=${2:-"error"}

COUNT=0
```

```

if [ ! -f "$LOGFILE" ]; then
echo "File not found. Trying fallbacks..."
for FALLBACK in /var/log/syslog /var/log/messages; do
[ -f "$FALLBACK" ] && LOGFILE=$FALLBACK && break
done
fi

while IFS= read -r LINE; do
if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))
fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo "--- Last 5 matches ---"
grep -i "$KEYWORD" "$LOGFILE" | tail -5

```

Script 5 — Open Source Manifesto Generator

Shell Concepts Used: read for user input, string concatenation, writing to file with > and >>, date command, aliases (commented)

Explanation:

This script interactively prompts the user with three questions using the read command, capturing their answers into variables. It then uses string concatenation to compose a personalised open-source philosophy statement from those answers. The manifesto is written to a text file named after the current user using > (create/overwrite) and >> (append). The date command provides a timestamp. The script also includes a comment demonstrating the alias concept — how you might create a shell alias to invoke the script quickly. The output is then displayed using cat.

Script Code:

```

#!/bin/bash

# Script 5: Open Source Manifesto Generator

# Alias concept: alias manifest='bash script5_manifesto_generator.sh'

echo "Answer three questions to generate your manifesto."
read -p "1. One open-source tool you use daily: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "Open Source Manifesto – $DATE" > "$OUTPUT"

```

```
echo "By: $(whoami)" >> "$OUTPUT"
```

```
echo "" >> "$OUTPUT"
```

```
echo "Every day I rely on $TOOL – a tool I did not pay for." >> "$OUTPUT"
```

```
echo "To me, freedom means $FREEDOM. GPL makes that real." >> "$OUTPUT"
```

```
echo "I would build $BUILD and release it openly." >> "$OUTPUT"
```

```
echo "Open source is civilization's best model for progress." >> "$OUTPUT"
```

```
echo "Manifesto saved to $OUTPUT"
```

```
cat "$OUTPUT"
```

Conclusion

Auditing Git reveals something that is easy to overlook when you use a tool every day: that it is not inevitable. Git exists because a proprietary tool failed a community, and one person — already overloaded with maintaining the Linux kernel — spent ten days building a replacement and releasing it freely. That decision, and the GPL v2 license that protects it, is why every software developer in the world now has access to the same professional-grade version control system, regardless of whether they work at a billion-dollar company or in a bedroom.

The FOSS ecosystem around Git — GitHub, GitLab, CI/CD pipelines, GitOps, the pull request model of collaboration — is one of the most consequential technology stacks ever assembled, and it was built on a foundation of freely shared code. Understanding that is not a peripheral concern for a software engineer. It is the context in which every line of professional code is written.

References

- GNU Project. The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition. opensource.org/osd
- Raymond, Eric S. The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Torvalds, Linus. Git initial commit message. April 7, 2005. github.com/git/git
- Git SCM. Git Reference Manual. git-scm.com/docs
- SPDX License List — GPL-2.0-only. spdx.org/licenses/GPL-2.0-only.html
- Linux Command Line, William Shotts. linuxcommand.org
- Hamano, Junio C. Git Mailing List Archive. lore.kernel.org/git/